



Write Pretty

Personal Project 02

Jun Xiang

30th September 2026

Table of Contents

1. Overview	3
1.1 Problem	3
1.2 Solution	3
1.3 Goals	4
1.4 Features	4
2. Scope	5
2.1 In Scope	5
2.2 Out of Scope	6
3. Requirements	6
3.1 Functional Requirements	6
3.2 Non-Functional Requirements	7
4. Setup & Design	7
4.1 Technology Stack	7
4.2 System Architecture	7
4.3 Design Decisions	7
5. Limitations & Risks	8
6. Testings and Findings	9
6.1 Test Case 1: General Text Cleanup	9
6.2 Test Case 2: Markdown & Code Preservation	10
6.3 Test Case 3: URL & Email Handling	11
6.4 Test Case 4: History Mode	12
6.5 Test Case 5: Text Statistics, Controls & Scrolling	14
6.6 Test Case 6: Local Processing	16
6.6 Test Case 7: Local Autosave	17
6.6 Test Case 8: Workspace Reset	18
7. Challenges	19
8. Improvements	20

1. Overview

WritePretty is a lightweight text cleanup tool that improves the consistency and readability of written text using simple, rule-based processing.

It can clean spacing and punctuation, correct clear capitalisation issues, remove selected Markdown formatting, and use the browser's built-in spellcheck to help identify possible spelling mistakes.

WritePretty does not use AI to rewrite the user's text. Its purpose is to make small, predictable improvements while preserving the user's original wording.

1.1 Problem

Many users write content in plain text or Markdown but spend extra time manually fixing:

- spelling mistakes
- inconsistent capitalisation
- spacing and punctuation issues
- unnecessary line breaks
- unwanted Markdown formatting

Many existing tools focus on grammar improvement or AI rewriting when only simple, predictable cleanup may be needed.

1.2 Solution

WritePretty is a lightweight text formatting and correction tool that improves text consistency and readability without using AI. The tool:

- corrects clear spacing, punctuation, and capitalisation issues
- allows users to enable or disable individual cleanup rules
- optionally removes selected Markdown formatting and extra line breaks
- uses the browser's built-in spellcheck for spelling detection
- protects code, URLs, and email addresses from normal cleanup
- preserves the user's original wording

1.3 Goals

The main goals of WritePretty are to:

- reduce time spent on simple text cleanup
- keep text processing easy to understand
- produce predictable results
- preserve the user's original wording and structure
- give users control over which cleanup rules are applied
- keep user text private by processing it locally

1.4 Features

Test Input	Description
Text Cleanup	Removes unnecessary spacing and fixes clear punctuation and capitalisation issues.
Processing Controls	Allows users to enable or disable cleanup rules, including optional Markdown and line-break removal.
Browser Spellcheck	Uses the browser's built-in spellcheck to identify possible spelling mistakes.
Protected Content	Avoids changing explicit URLs, email addresses, inline code, and code blocks.
Side-by-Side View	Shows the Original and Pretty text together for easy comparison.
History Mode	Stores up to five text entries and allows users to switch between or combine them.
Synchronised Scrolling	Keeps the Original and Pretty panels at approximately the same scroll position.
Text Statistics	Displays word and character counts for both Original and Pretty text.
Help & Tooltips	Provides built-in guidance and explanations for buttons and controls.
Copy & Download	Allows the processed text to be copied or downloaded as a text file.
Responsive Layout	Adapts the interface for smaller screens and mobile devices.
Local Autosave	Saves the current text, History Mode entries, active history entry, and cleanup settings in the browser and restores them when the page is reopened.

Test Input	Description
Workspace Reset	Clears saved text, history, and cleanup settings and restores WritePretty to its default state after confirmation.
Local Processing	Processes user text directly in the browser.

2. Scope

2.1 In Scope

- English text input and processing
- spacing and punctuation cleanup
- clear capitalisation correction
- browser-based spelling detection
- user-selectable cleanup controls
- optional removal of extra line breaks
- optional removal of selected Markdown formatting
- protection of code, explicit URLs, and email addresses
- side-by-side comparison
- History Mode
- combined history preview
- synchronised scrolling
- word and character counts for Original and Pretty text
- copying processed text
- downloading processed text
- Help guidance and tooltips
- responsive layout for smaller screens
- local autosave of text, History Mode entries, and cleanup settings
- restoration of the previous workspace after reopening or refreshing the page
- local browser-based processing

2.2 Out of Scope

The current version does not provide:

- AI-based rewriting
- advanced grammar correction
- automatic spelling correction
- content generation
- plagiarism detection
- languages other than English
- cloud document storage
- user accounts

3. Requirements

3.1 Functional Requirements

- Allow users to enter or paste text
- Process text automatically as the user types
- Correct clear spacing, punctuation, and capitalisation issues
- Allow individual cleanup rules to be enabled or disabled
- Allow optional removal of extra line breaks
- Allow optional removal of selected Markdown formatting
- Use the browser's built-in spellcheck for spelling detection
- Protect inline code, fenced code blocks, indented code, explicit URLs, and email addresses
- Display the original and processed text side by side
- Display word and character counts for both Original and Pretty text
- Allow up to five History Mode entries
- Allow users to switch between, delete, and combine history entries
- Keep both text panels synchronised while scrolling
- Allow users to copy processed text
- Allow users to download processed text as a .txt file
- Provide Help information and tooltips
- Adapt the interface for smaller screens
- Automatically save the current text, History Mode entries, active history entry, and cleanup settings in the browser
- Restore the saved workspace after refreshing or reopening the page
- Allow users to reset text, history, and cleanup settings to their defaults
- Require confirmation before performing a workspace reset

3.2 Non-Functional Requirements

- Respond quickly for normal text inputs
- Have a simple interface
- Process user text locally
- Give the same result for the same input and settings
- Avoid unnecessary changes to user content
- Remain easy to maintain and extend
- Work in modern web browsers
- Remain usable on smaller screen sizes

4. Setup & Design

4.1 Technology Stack

WritePretty is a client-side web application built with:

- HTML for application structure
- CSS for styling and responsive behaviour
- JavaScript for text processing and interaction
- the browser's built-in spellcheck for spelling detection
- the Web Storage API (localStorage) for local workspace persistence

4.2 System Architecture

- WritePretty uses a simple browser-based processing flow: User Input → Selected Cleanup Rules → Rule-Based Processing → Processed Output → User Review
- Text cleanup is performed in the browser and updates when the user types or changes a cleanup control.
- Browser spellcheck operates independently from WritePretty's cleanup rules.
- Workspace state is stored locally in the browser so text, History Mode entries, and cleanup settings can be restored between sessions.

4.3 Design Decisions

- User text is processed in the browser instead of being sent to an AI rewriting service.
- Rule-based processing is used to produce predictable and consistent results.
- Only clear issues are changed automatically. Text is left unchanged when the intended correction is uncertain.
- Spelling detection is handled by the browser instead of a custom spelling system.
- Spelling mistakes are not automatically replaced because the intended word may not always be clear.
- Cleanup rules can be individually enabled or disabled so users control how their text is processed.

- Markdown formatting and extra line breaks are preserved by default and are only removed when the corresponding controls are enabled.
- Code, URLs, and email addresses are protected where possible so cleanup rules do not accidentally change meaningful content.
- The Original and Pretty panels allow users to review changes before copying or downloading the result.
- Workspace data is saved using browser local storage so autosave can work without requiring user accounts or cloud storage.
- Reset requires confirmation because it permanently clears the locally saved workspace and restores the default settings.

5. Limitations & Risks

WritePretty has several limitations due to its rule-based approach:

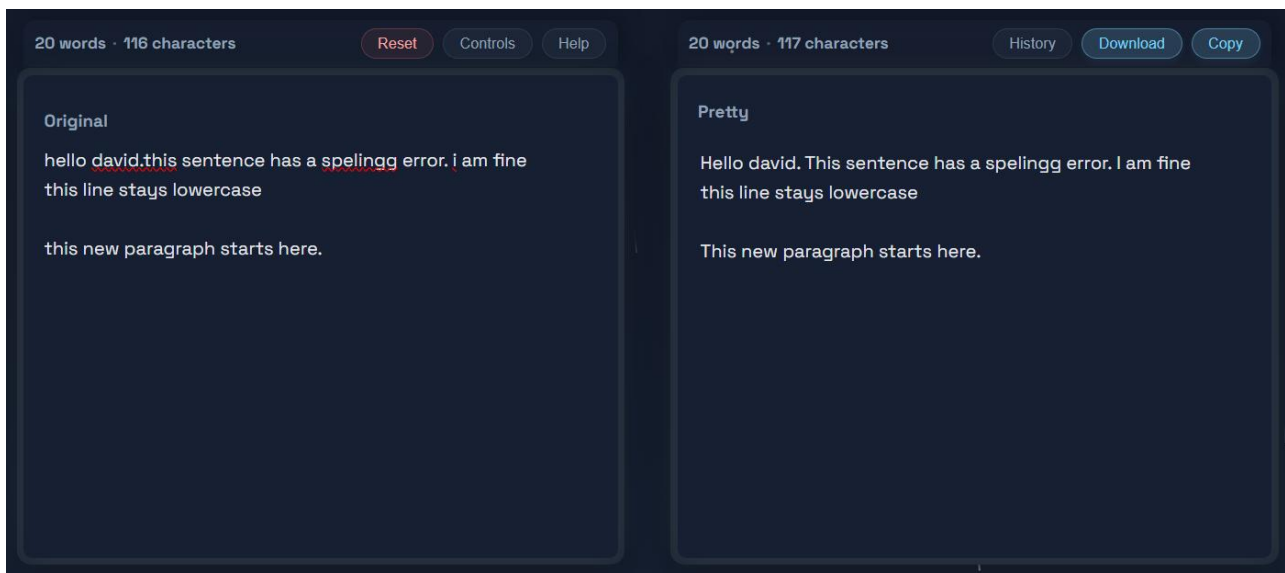
- The system cannot understand sentence meaning or context as deeply as a grammar or AI writing tool.
- Complex grammar errors are outside the current scope.
- Only English is currently supported.
- Spelling detection depends on the browser and may vary between browsers, operating systems, and user settings.
- Names, brands, technical terms, or uncommon words may be marked as misspelled by the browser.
- Bare domains such as “example.com” may sometimes be treated like normal punctuation because they do not contain an explicit URL prefix.
- Code is protected from cleanup, which means issues inside protected code are intentionally left unchanged.
- Optional Markdown and line-break removal can change document structure when enabled.
- A single line break by itself is not treated as the start of a new sentence.
- Rule-based processing may occasionally make an incorrect change or fail to detect an issue.
- Autosaved workspace data is stored only in the current browser and may be lost if browser storage is cleared.

6. Testings and Findings

6.1 Test Case 1: General Text Cleanup

Test Input	Expected Result	Findings
hello david.this sentence has a spelingg error. i am fine this line stays lowercase this new paragraph starts here.	Hello david. This sentence has a spelingg error. I am fine this line stays lowercase This new paragraph starts here.	• Clear spacing and capitalisation issues were corrected. • The misspelled word was not automatically replaced. • Line and paragraph structure was preserved.

Test Evidence



6.2 Test Case 2: Markdown & Code Preservation

Test Input	Expected Result	Findings
<pre># hello world **this is important.** - first item - second item Use `const name = "david";` here. ```js const name = "david"; let number = 5;</pre>	<pre># Hello world **This is important.** - First item - Second item Use `const name = "david";` here. ```js const name = "david"; let number = 5;</pre>	• Markdown structure was preserved while Markdown-removal controls were disabled. • Readable Markdown text was cleaned and capitalised. • Inline and fenced code remained unchanged. • Formatting inside protected code was not modified.

Test Evidence

The screenshot displays a side-by-side comparison of a Markdown document. On the left, the 'Original' version (21 words, 163 characters) shows the raw Markdown syntax. On the right, the 'Pretty' version (21 words, 156 characters) shows the rendered HTML output. The 'Pretty' version correctly interprets the Markdown: the heading is H1, bold text is bold, list items are bulleted, code blocks are fenced, and code is syntax-highlighted. The interface includes buttons for 'Reset', 'Controls', 'Help', 'History', 'Download', and 'Copy'.

21 words · 163 characters

Reset Controls Help

Original

```
# hello world

**this is important.**
- first item
- second item

Use `const name = "david";` here.

```js
const name = "david";
let number = 5;
```

21 words · 156 characters

History Download Copy

Pretty

```
Hello world

This is important.
- First item
- Second item

Use `const name = "david";` here.

```js
const name = "david";
let number = 5;
```

6.3 Test Case 3: URL & Email Handling

Test Input	Expected Result	Findings
Visit Https://example.com/MyFile today.	Visit https://example.com/MyFile today.	• Explicit https:// URLs were protected. • www. URLs were protected. • Email addresses were preserved. • The URL scheme was normalised from Https:// to https:// • Bare domains remain a known limitation.
Go to www.example.com/help for more information.	Go to www.example.com/help for more information.	
Email test@example.com for help.	Email test@example.com for help.	
Visit example.com today.	Visit example. Com today.	

Test Evidence

25 words · 149 characters

ResetControlsHelp

Original

Visit Htps://example.com/MyFile today.

Go to www.example.com/help for more information.

Email test@example.com for help.

Visit example.com today.

25 words · 150 characters

HistoryDownloadCopy

Pretty

Visit https://example.com/MyFile today.

Go to www.example.com/help for more information.

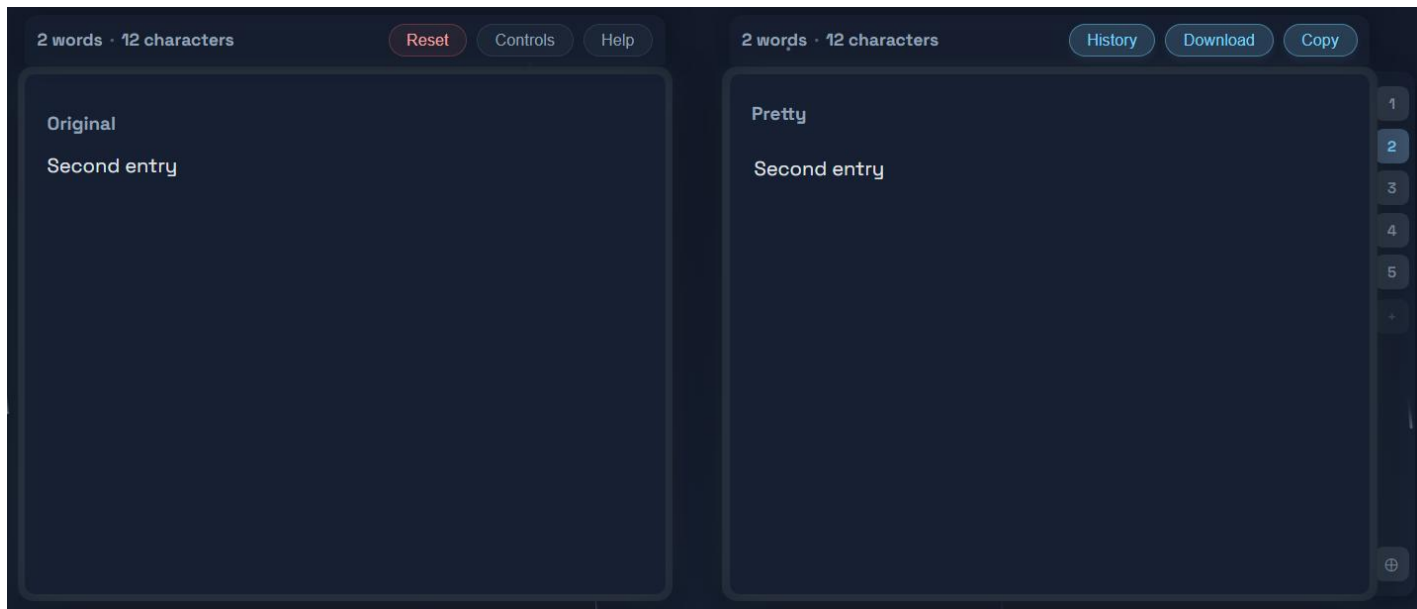
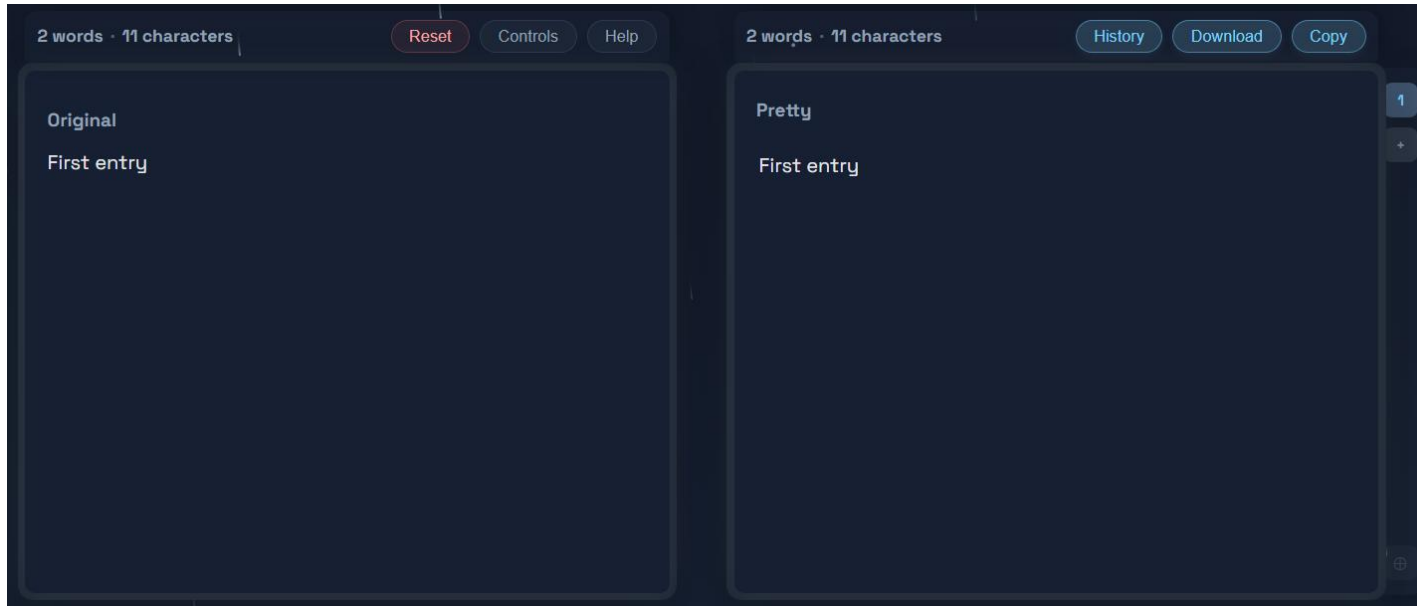
Email test@example.com for help.

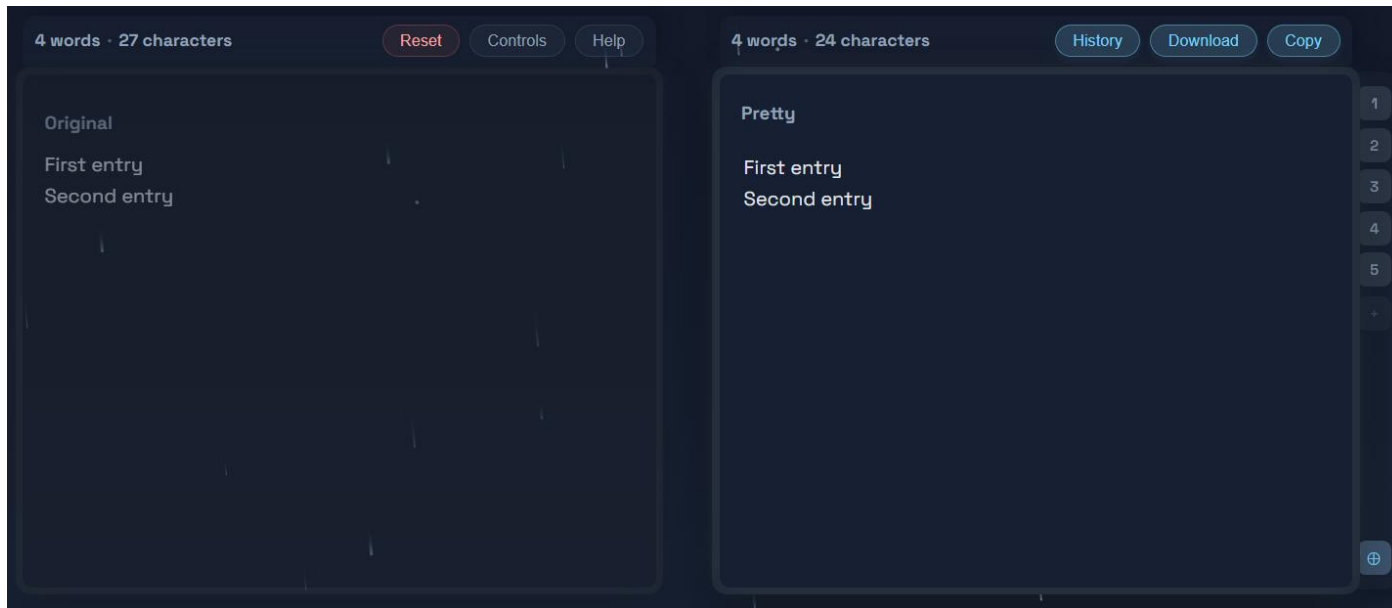
Visit example. Com today.

6.4 Test Case 4: History Mode

Test Steps	Expected Result	Findings
1. Type "First entry" 2. Turn History on. 3. Confirm tab 1 appears with "First entry" 4. Click '+' once. 5. Type "Second entry" 6. Confirm tab 1 and tab 2 contain the correct text. 7. Click '+' until there are 5 entries, and confirm '+' becomes disabled. 8. Click '⊕' and confirm all entries appear together. 9. While in Combined View, delete one history entry and confirm the combined text updates while the remaining entries are still shown. 10. Click ⊕ again to return to the previously active entry.	Each history entry keeps its own text. Users can switch between entries, view them together, and delete entries. A maximum of five entries can be created.	• History entries were stored separately. • Switching tabs restored the correct text. • Combined View displayed all entries together. • Deleting an entry in Combined View updated the combined text correctly. • Returning from Combined View restored the active entry. • The '+' button was disabled after five entries.

Test Evidence

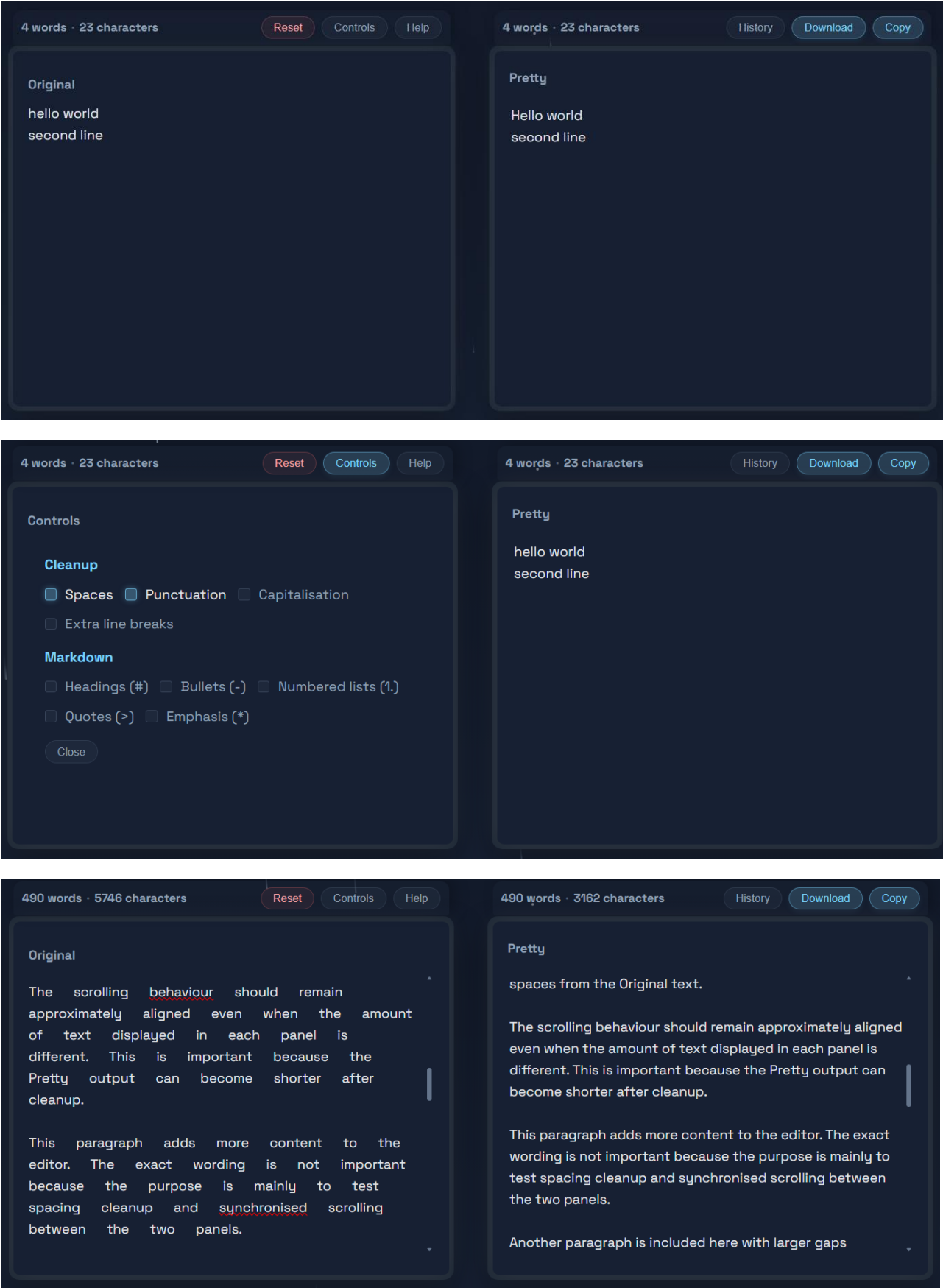




6.5 Test Case 5: Text Statistics, Controls & Scrolling

Test Input	Expected Result	Findings
1. Type "hello world." and add a new line with "Second line." 2. Confirm the Original and Pretty word and character counts. 3. Open Controls. 4. Disable Capitalisation and confirm the Pretty output updates immediately. 5. Re-enable Capitalisation and confirm the Pretty output updates again. 6. Enter enough text to make both panels scrollable. 7. Scroll either panel and confirm the other panel stays approximately aligned.	Both Original and Pretty display the correct word and character counts. Changing a cleanup control immediately updates the Pretty output. Original and Pretty remain approximately aligned while scrolling.	• Original and Pretty word and character counts updated correctly. • Cleanup controls changed the Pretty output immediately. • Scrolling either panel kept the other panel aligned proportionally.

Test Evidence



6.6 Test Case 6: Local Processing

Test Input	Expected Result	Findings
1. Open Developer Tools and select the Network tab. 2. Clear the existing network log. 3. Type or paste text into WritePretty. 4. Edit the text several times and use the cleanup features. 5. Check whether typing or processing creates any new requests containing the user's text.	Text cleanup should happen directly in the browser without sending the entered text to an external processing service.	• Text processing was performed locally in the browser. • Typing and editing did not send the entered text to an external text-processing service. • No AI or server-based rewriting requests containing the user's input were observed.

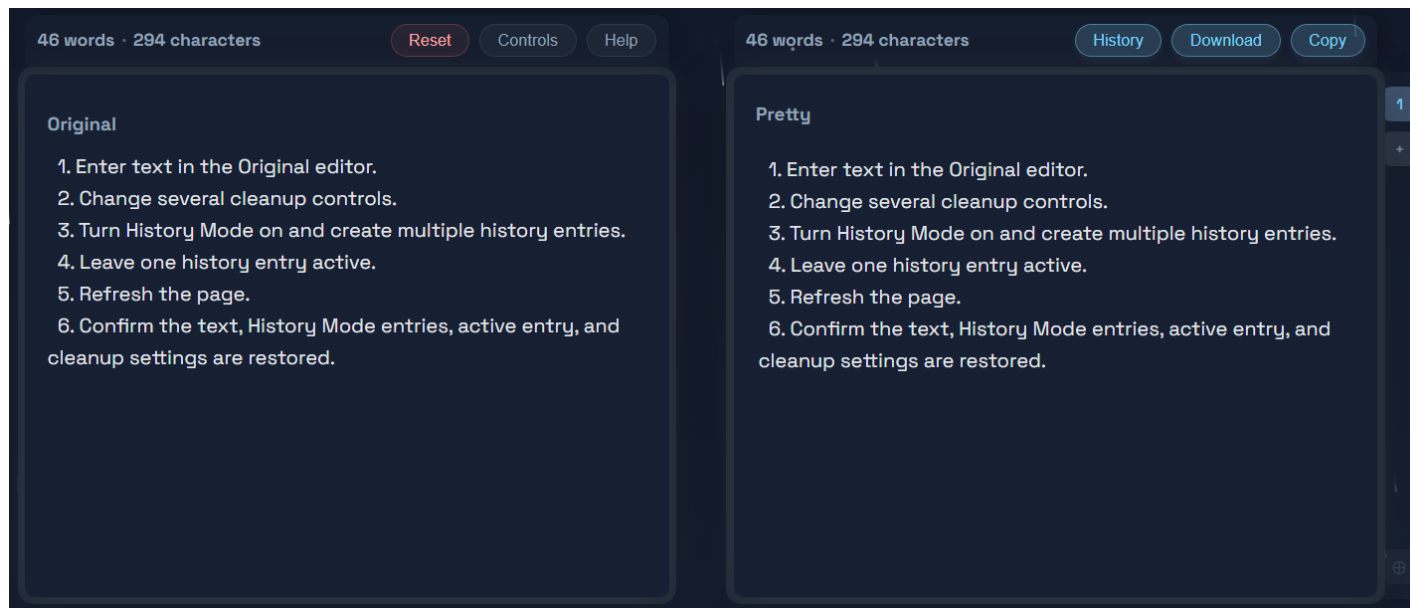
Test Evidence

The screenshot shows the WritePretty application interface on the left and the browser's developer tools network tab on the right. The application interface has a dark theme and displays the text: "Keep your words. Make it pretty." followed by "A simple rule-based text cleanup tool that improves consistency and readability." Below this, it shows "47 words · 288 characters" and buttons for "Reset", "Controls", "Help", "History", "Download", and "Copy". The "Original" text area contains the test steps, and the "Pretty" text area shows the same text after processing. The browser's developer tools network tab is open, showing a list of requests. The first request is "writepretty/" with a status of 200, type of document, and size of 3.0 kB. Subsequent requests for "style.css", "script.js", and "css2?family=Space+Grotesk..." all have a status of 200 and are served from the local disk. The bottom status bar of the developer tools indicates "5 requests · 3.1 kB transferred · 90.7 kB resources · Finish: 4.10 s · DOMContentLoaded: 4.11 s · Load: 4.12 s".

6.6 Test Case 7: Local Autosave

Test Input	Expected Result	Findings
1. Enter text in the Original editor. 2. Change several cleanup controls. 3. Turn History Mode on and create multiple history entries. 4. Leave one history entry active. 5. Refresh the page. 6. Confirm the text, History Mode entries, active entry, and cleanup settings are restored.	The previous workspace is restored after refresh, including text, History Mode entries, the active history entry, and cleanup settings.	• Text was restored after refresh. • History Mode entries were restored. • The previously active history entry was restored. • Cleanup settings were restored. • Pretty output and text statistics were regenerated correctly.

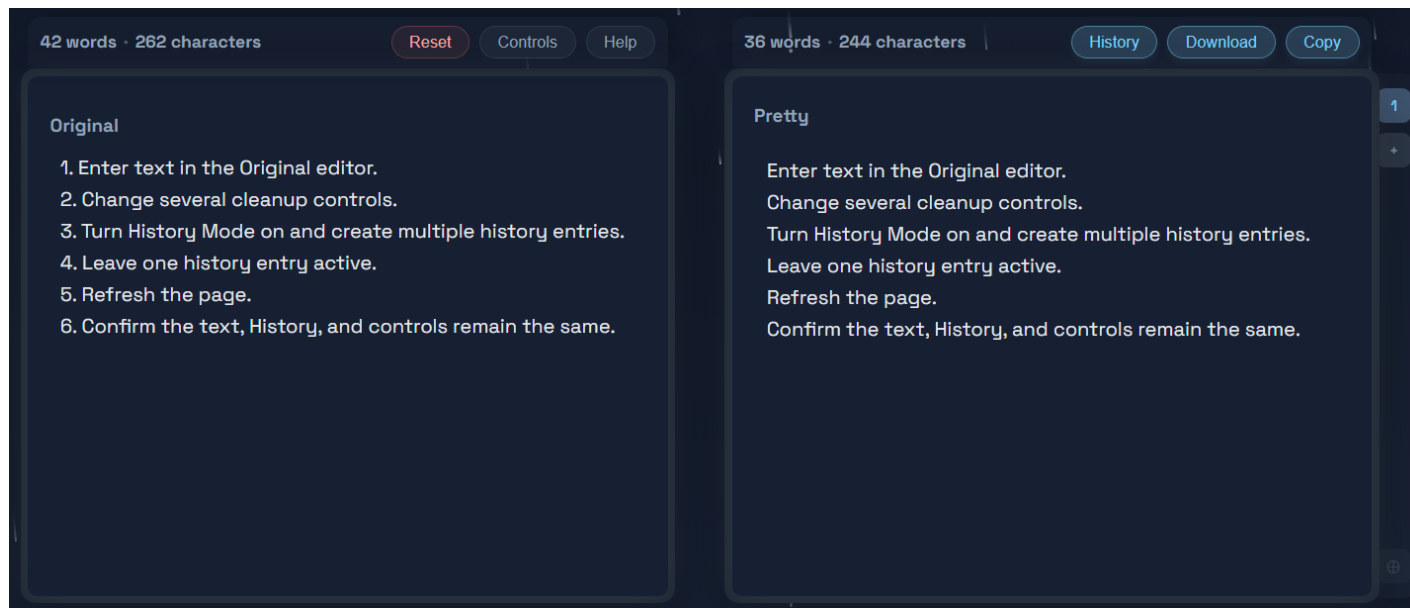
Test Evidence



6.6 Test Case 8: Workspace Reset

Test Input	Expected Result	Findings
1. Enter text in the Original editor. 2. Change several cleanup controls. 3. Turn History Mode on and create multiple history entries. 4. Leave one history entry active. 5. Refresh the page. 6. Confirm the text, History, and controls remain the same.	The previous workspace is restored after refresh, including text, History Mode entries, the active history entry, and cleanup settings.	• Text was restored after refresh. • History Mode entries were restored. • The previously active history entry was restored. • Cleanup settings were restored back to default. • Pretty output and text statistics were regenerated correctly.

Test Evidence



7. Challenges

Challenges	How it was handled
It can be difficult to know when capitalisation is definitely wrong.	Only clear cases are corrected automatically, while uncertain cases are left unchanged.
The original custom spelling and highlighting system became too complex for the scope of the project.	The implementation was simplified by using the browser's built-in spellcheck instead.
Normal cleanup rules can accidentally change Markdown or code.	Code and other protected content are temporarily excluded from normal cleanup.
It was difficult to tell whether unusual spacing was accidental or intentional.	Conservative formatting rules are used, while more structural cleanup is kept optional.
Bare domains such as “example.com” can look like punctuation mistakes.	Explicit URLs and email addresses are protected, while bare domains remain a known limitation.

8. Improvements

- **Custom Terms**
 - Let users define preferred spellings or capitalisation for names, brands, and technical terms.
 - Detect inconsistent use of those terms.
- **Spelling Suggestions**
 - Show possible replacements for spelling mistakes while leaving the final choice to the user.
- **Additional Cleanup Rules**
 - Add more optional cleanup rules where they can be applied predictably.
- **Accessibility Improvements**
 - Continue improving keyboard and screen-reader support.
 - Improve tooltip behaviour on touch devices where needed.